

IP Kameraların Mobil Uygulamada Canlı İzlenmesi

WebRTC Tabanlı Statik IP'siz Çözüm Mimarisi

Hazırlayan: Yavuzhan Kursun

Tarih: Mart 2026

1. Giriş ve Problem Tanımı

Günlük yaşamda ve endüstride IP kameraların kullanımı hızla yaygınlaşmaktadır. Güvenlik, uzaktan izleme, üretim hattı takibi gibi pek çok senaryoda IP kameralardan gelen canlı görüntünün mobil cihazlardan izlenebilmesi kritik bir ihtiyaçtır. Ancak bu ihtiyacı karşılamak, görüldüğü kadar basit değildir.

1.1 Temel Zorluklar

- IP kameraların çoğu RTSP (Real-Time Streaming Protocol) ile yayın yapar; ancak RTSP, web tarayıcıları ve mobil uygulamalar tarafından doğrudan desteklenmez.
- Statik IP adresi olmayan ağlarda kameralara dışarıdan erişmek, NAT (Network Address Translation) ve firewall engelleri nedeniyle oldukça güçtür.
- Geleneksel çözümler (port forwarding, DDNS) hem güvenlik riski oluşturur hem de dinamik IP ortamlarında kararlı çalışmaz.
- HLS gibi HTTP tabanlı protokoller düşük gecikme gerektiren senaryolarda yetersiz kalır (genellikle 5–30 saniye gecikme).

Bu doküman, WebRTC teknolojisi kullanarak statik IP'ye ihtiyaç duymadan IP kamera görüntülerini mobil uygulamalarda düşük gecikmeyle nasıl izletebileceğimizi açıklamaktadır.

2. WebRTC Nedir?

WebRTC (Web Real-Time Communication), web tarayıcıları ve mobil uygulamalar arasında gerçek zamanlı ses, video ve veri iletişimi sağlayan açık kaynak bir protokol setidir. Google tarafından geliştirilmiş olup tüm modern tarayıcılar (Chrome, Firefox, Safari, Edge) ve mobil platformlar (iOS, Android) tarafından desteklenir.

2.1 WebRTC'nin Temel Bileşenleri

Bileşen	Açıklama
ICE	Interactive Connectivity Establishment – Bağlantı kurulumu için en uygun yolu bulan çerçeve
STUN	Session Traversal Utilities for NAT – Cihazın kamusal IP adresini keşfeden hafif protokol
TURN	Traversal Using Relays around NAT – Doğrudan bağlantı kurulamadığında medyayı aktaran relay sunucu
SDP	Session Description Protocol – Codec, çözünürlük gibi medya özelliklerini tanımlayan standart
SRTP	Secure RTP – Medya verilerini şifreli olarak taşıyan protokol (WebRTC'de zorunlu)
Signaling	Peer'ler arasında SDP ve ICE bilgilerinin değişimi için kullanılan sinyal kanalı (WebSocket vb.)

2.2 Neden WebRTC?

- Düşük gecikme: 0.5 saniyenin altında (sub-second) gecikme ile gerçek zamanlı izleme mümkün.
- Eklenti gerektirmez: Tarayıcı ve mobil SDK'lar üzerinden çalışır.
- Yerleşik güvenlik: Tüm medya trafiği SRTP ile şifrelenir.
- NAT geçişi: ICE/STUN/TURN mekanizmaları ile statik IP'ye gerek kalmadan bağlantı kurulur.
- Geniş platform desteği: Chrome, Safari, Firefox, Android, iOS üzerinde çalışır.

3. Statik IP Olmadan NAT Geçişi

Statik IP'siz senaryolarda en büyük zorluk, her iki tarafın da NAT/firewall arkasında olmasıdır. WebRTC, ICE frameworkü aracılığıyla bu sorunu sistematik olarak çözer.

3.1 ICE (Interactive Connectivity Establishment)

ICE, iki peer arasında mümkün tüm bağlantı yollarını (candidate) keşfeder ve en verimli olanı seçer. Üç tür candidate vardır:

1. Host Candidate: Cihazın kendi yerel (LAN) IP adresi.
2. Server Reflexive (srflx): STUN sunucusundan öğrenilen kamusal IP adresi.
3. Relay Candidate: TURN sunucusu üzerinden geçen yedek bağlantı.

3.2 STUN Sunucusu

STUN (Session Traversal Utilities for NAT) sunucusu, NAT arkasındaki cihazın kamusal IP adresini ve port numarasını keşfetmesini sağlar. Cihaz, STUN sunucusuna bir istek gönderir; sunucu da cihazın dışarıdan görünen adresini yanıt olarak döner. STUN sunucuları medya trafiği taşımaz, yalnızca adres keşfi yapar.

Google'ın ücretsiz STUN sunucusu (stun:stun.l.google.com:19302) test ortamları için yaygın olarak kullanılır. Bağlantıların yaklaşık %70–80'i yalnızca STUN ile kurulabilir.

3.3 TURN Sunucusu

Symmetric NAT veya katı firewall kuralları nedeniyle doğrudan bağlantı kurulamadığında TURN (Traversal Using Relays around NAT) devreye girer. TURN sunucusu bir relay görevi görür: tüm medya trafiği TURN sunucusu üzerinden aktarılır. Bu nedenle STUN'a göre daha fazla bant genişliği ve işlem gücü tüketir. WebRTC konferanslarının yaklaşık %20–30'u TURN sunucusu gerektirir.

Açık kaynak TURN sunucu yazılımı olarak coturn yaygın şekilde kullanılır. Bulut servis sağlayıcılarında (AWS, DigitalOcean, Hetzner) kolayca deploy edilebilir.

3.4 Bağlantı Akışı Özeti

1. Kamera tarafındaki gateway/proxy, STUN sunucusuna istek gönderir ve kamusal IP'sini öğrenir.
2. Mobil uygulama da aynı işlemi yapar.

3. Signaling sunucusu üzerinden SDP (codec bilgileri) ve ICE candidate'ler karşılıklı değişir.
4. ICE, en uygun bağlantı yolunu seçer (doğrudan veya TURN relay).
5. SRTP ile şifrelenmiş medya akışı başlar.

4. Sistem Mimarisi

IP kamera görüntülerinin mobil uygulamada izlenmesi için önerilen mimari aşağıdaki bileşenlerden oluşur:

4.1 Genel Mimari

Katman	Detay
IP Kamera	RTSP/H.264 akışı üretir. Yerel ağda çalışır.
Media Gateway	RTSP'yi WebRTC'ye dönüştürür. Kamera ile aynı LAN'da çalışır (go2rtc, Janus, Kurento, MediaMTX).
Signaling Server	SDP ve ICE candidate değişimini koordine eder (WebSocket/HTTP). Bulutta veya kendi sunucunuzda çalışır.
STUN/TURN	NAT geçişi için adres keşfi ve relay hizmeti sunar.
Mobil Uygulama	WebRTC client olarak canlı görüntüyü alır ve oynatır (React Native / Native SDK).

4.2 Veri Akışı

[IP Kamera] → RTSP/H.264 → [Media Gateway (go2rtc)] → WebRTC → [STUN/TURN] → [Mobil Uygulama]

Media Gateway, kameranın RTSP akışını alır ve H.264 codec'ini WebRTC uyumlu hale getirir. Bu işlem genellikle transkod gerektirmez çünkü WebRTC modern tarayıcılarda H.264'ü doğrudan destekler. Transkod gerektiğinde (VP8/VP9 dönüşümü) sunucu tarafında önemli işlem gücü harcanabilir.

5. Açık Kaynak Araçlar ve Teknolojiler

5.1 go2rtc

go2rtc, Go diliyle yazılmış hafif ve çok yönlü bir kamera akış uygulamasıdır. RTSP, RTMP, HTTP-FLV, WebRTC, MSE, HLS, MJPEG ve HomeKit gibi çok sayıda protokolü destekler. Raspberry Pi gibi düşük güçlü cihazlarda bile verimli çalışır.

- RTSP'den WebRTC'ye otomatik dönüşüm
- ~0.5 saniye gecikme ile canlı izleme
- Hikvision, Dahua, Tapo, Tuya gibi pek çok marka desteği
- Home Assistant ve Frigate ile entegrasyon
- Docker ile kolay kurulum

- ICE server (STUN/TURN) yapılandırma desteği

5.2 Janus WebRTC Gateway

Janus, çok amaçlı bir açık kaynak WebRTC gateway'dir. Plugin mimarisi sayesinde video konferans, akış izleme, SIP entegrasyonu gibi farklı senaryolara adapte edilebilir. RTSP akışlarını FFmpeg ile alıp WebRTC'ye dönüştürebilir.

5.3 MediaMTX (eski adıyla rtsp-simple-server)

Go ile yazılmış, RTSP/RTMP/HLS/WebRTC destekleyen minimalist bir medya sunucusudur. Tek binary olarak çalışır, yapılandırması basittir.

5.4 coturn (TURN Sunucusu)

coturn, en yaygın açık kaynak STUN/TURN sunucu implementasyonudur. Dinamik IP senaryolarında NAT geçişi için kritik öneme sahiptir. Ubuntu/Debian üzerinde apt ile kolayca kurulabilir.

5.5 Teknoloji Karşılaştırma Tablosu

	go2rtc	Janus	MediaMTX	Wowza
Lisans	MIT	GPLv3	MIT	Ticari
Dil	Go	C	Go	Java
Kaynak Tüketimi	Çok Düşük	Düşük	Çok Düşük	Yüksek
Kurulum	Tek Binary	Karmaşık	Tek Binary	Kolay
RTSP→WebRTC	Evet	FFmpeg ile	Evet	Evet

6. Mobil Uygulama Entegrasyonu

6.1 React Native ile WebRTC

React Native/Expo ortamında WebRTC entegrasyonu için react-native-webrtc kütüphanesi kullanılır. Bu kütüphane, native WebRTC API'lerine erişim sağlar ve Expo config plugin ile uyumludur.

Kurulum: `npx expo install react-native-webrtc @config-plugins/react-native-webrtc`

react-native-webrtc, RTCPeerConnection, RTCSessionDescription ve RTCIceCandidate gibi temel WebRTC API'lerini JavaScript tarafında kullanıma sunar. Ancak yalnızca istemci tarafını sağlar; sinyal sunucusu ve medya gateway'i ayrıca kurulmalıdır.

6.2 Mobil Uygulama Akışı

1. Uygulama açıldığında Signaling Server'a WebSocket bağlantısı kurulur.
2. Kullanıcı izlemek istediği kamerayı seçer.

3. Signaling Server, Media Gateway'e "bu kullanıcı bu kamerayı izlemek istiyor" mesajını iletir.
4. SDP Offer/Answer ve ICE candidate değişimi gerçekleşir.
5. WebRTC peer bağlantısı kurulur, video akışı başlar.

6.3 Alternatif: WebView Yaklaşımı

Daha basit bir yaklaşım olarak, go2rtc veya benzeri bir gateway'in sunduğu WebRTC oynatıcı HTML sayfası doğrudan bir WebView içinde açılabilir. Bu yöntem, native WebRTC entegrasyonuna göre daha hızlı geliştirme süresi sunar ancak performans ve UX açısından sınırlıdır.

7. Pratik Senaryo: Dinamik IP ile Uzaktan Kamera İzleme

Aşağıda, statik IP'si olmayan bir lokasyondaki IP kameranın mobil uygulamadan izlenmesi için adım adım uygulama senaryosu anlatılmaktadır.

7.1 Gerekli Altyapı

- Lokasyonda: IP kamera (RTSP destekli) + go2rtc çalıştıran bir cihaz (Raspberry Pi, mini PC veya mevcut bilgisayar)
- Bulutta: Signaling Server (Node.js + WebSocket) + coturn (STUN/TURN) – ufak bir VPS yeterlidir
- Kullanıcı tarafı: React Native mobil uygulama

7.2 go2rtc Yapılandırma Örneği

go2rtc.yaml dosyasında kamera tanımı ve ICE server ayarı şu şekilde yapılır:

```
api: listen ":1984" | streams: ofis_kamera: rtsp://admin:sifre@192.168.1.100/main | webrtc: listen ":8555", ice_servers: [urls: stun:stun.l.google.com:19302]
```

7.3 TURN Sunucusu Gerekliliği

Dinamik IP ortamında, özellikle mobil veri (4G/5G) kullanan istemciler için TURN sunucusu olmazsa olmazdır. Mobil operatörler genellikle Symmetric NAT kullanır ve STUN tek başına bağlantı kurmaya yetmez. coturn, bulutta çalışan TURN sunucusu olarak bu sorunu çözer.

8. Alternatif Yaklaşımlar

8.1 P2P Kamera Teknolojisi

Modern IP kameraların çoğu, üretici tarafından sağlanan P2P bulut servisi ile gelir. Kamera, internete bağlandığında benzersiz bir ID ile üretici sunucusuna kaydolar. Kullanıcı, üretici uygulaması üzerinden QR kod tarayarak kameraya erişir. Port forwarding veya statik IP gerekmez. Ancak bu yöntem, üretici ekosistemine bağımlılık yaratarak özelleştirme imkanını kısıtlar.

8.2 VPN Tüneli

WireGuard veya OpenVPN ile kamera lokasyonu ve izleme noktası arasında şifreli bir tünel kurulabilir. Bu yöntem güvenli olmakla birlikte, ek gecikme ve yapılandırma karmaşıklığı getirir.

8.3 DDNS (Dynamic DNS)

Dinamik IP adresini sabit bir alan adına bağlayan servistir (noip.com, duckdns.org vb.). Router veya yerel bir ajan, IP değiştiğinde DDNS kaydını günceller. Hala port forwarding gerektirir ve güvenlik riskleri taşır.

8.4 Yaklaşım Karşılaştırması

Yöntem	Gecikme	Statik IP	Güvenlik	Özelleştirme
WebRTC + TURN	<0.5s	Gerekmez	Yüksek (SRTP)	Tam kontrol
Üretici P2P	1–3s	Gerekmez	Değişken	Sınırlı
VPN + RTSP	1–5s	DDNS ile	Çok Yüksek	Tam kontrol
Port Forward	1–2s	Gerekir*	Düşük	Tam kontrol
HLS	5–30s	Sunucu gerekir	Orta	Orta

* DDNS ile dinamik IP kullanılabilir ancak port açma zorunludur.

9. Güvenlik Önerileri

- Tüm WebRTC bağlantıları SRTP ile şifrelenir; ek olarak Signaling Server'a WSS (WebSocket Secure) kullanın.
- TURN sunucusunda kimlik doğrulama (username/password veya token) mutlaka aktif olmalıdır.
- Kamera erişim bilgileri (RTSP URL'leri) asla istemci tarafına açılmamalıdır.
- Kameraları ana ağdan izole bir VLAN'a yerleştirin.
- go2rtc API'si için authentication aktif edin.
- HTTPS ve sertifika pinning kullanın.

10. Sonuç ve Öneriler

WebRTC, IP kamera görüntülerini statik IP olmadan mobil uygulamalara aktarmanın en etkili ve modern yöntemidir. ICE/STUN/TURN mekanizmaları sayesinde NAT ve firewall engelleri aşılar; SRTP sayesinde medya trafiği şifreli kalarak güvenlik sağlanır; sub-second gecikme ile gerçek zamanlı izleme mümkün olur.

Önerilen mimari, kamera lokasyonunda go2rtc gibi hafif bir media gateway, bulutta coturn (TURN) ve basit bir signaling server, mobil tarafta ise react-native-webrtc ile WebRTC entegrasyonu şeklindedir. Bu yapı, hem maliyet-etkin hem de ölçeklenebilir bir çözüm sunar.

Kaynaklar ve Referanslar

- WebRTC Resmi Belgeleri: webrtc.org
- go2rtc GitHub: github.com/AlexxIT/go2rtc

- MDN WebRTC Protokolleri:
developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- coturn GitHub: github.com/coturn/coturn
- react-native-webrtc: github.com/react-native-webrtc/react-native-webrtc
- Flashphoner RTSP to WebRTC: flashphoner.com